

FIFA 26 Football RAG Chatbot

Engineering Project Report

A Retrieval-Augmented Generation Chatbot for FIFA World Cup Knowledge

Project Type	Knowledge-Base RAG Chatbot
Domain	Football and FIFA World Cup
Frameworks	LangChain, ChromaDB, Chainlit, Groq, HuggingFace
Default Model	llama-3.3-70b-versatile (Groq)
Dataset Source	Public Wikipedia articles + curated World Cup summary file

1. Executive Summary

The FIFA 26 Football RAG Chatbot is a football-themed Retrieval-Augmented Generation system focused on FIFA World Cup knowledge. The project was designed to answer user questions using a grounded knowledge base rather than relying only on the internal memory of a language model. It combines public football data, text preprocessing, embeddings, vector search, reranking, strict prompting, source evidence, and a premium Chainlit interface.

The system retrieves relevant context from a local ChromaDB vector database, reranks candidate chunks for better source quality, and sends the best context to a Groq-powered LLM through LangChain. The chatbot also includes hallucination control, source confidence labels, evidence previews, and a user-friendly fallback response when the requested information is not available in the knowledge base.

2. Project Objectives

- Build a complete end-to-end RAG pipeline for football and FIFA World Cup knowledge.
- Use publicly available open-source knowledge sources with source metadata and attribution.
- Store cleaned and chunked data inside a local vector database.
- Generate grounded answers from retrieved context instead of unsupported model memory.
- Show source evidence with each answer to improve transparency.
- Add reranking and query expansion to improve retrieval quality.
- Control hallucination using a strict prompt and missing-information fallback.
- Design a polished football-themed chatbot interface using Chainlit, custom CSS, and custom JavaScript.

3. Scope of the Chatbot

The chatbot focuses on FIFA World Cup and football-related knowledge. The intended question coverage includes:

- FIFA World Cup winners and runner-ups.
- Final scorelines from historical World Cup finals.
- Hosts, tournament history, records, and milestones.
- Top scorers, Golden Boot winners, Golden Ball winners, Golden Glove winners, and young player awards.
- World Cup finals, qualification overview, mascots/trophies where available, and general football concepts.
- Future tournament status for the 2026 FIFA World Cup without making predictions.

The system does not claim to know facts that are not present in the dataset. For example, it should not predict the winner of the 2026 FIFA World Cup.

4. Technology Stack

Layer	Technology / Tool	Purpose
Language	Python	Main programming language
RAG Framework	LangChain	Prompting, model integration, RAG chain structure
Vector Database	ChromaDB	Local vector store for embedded chunks
Embeddings	HuggingFace all-MiniLM-L6-v2	Convert text into semantic vectors
Reranker	Cross-encoder reranker	Reorder retrieved chunks for better relevance
LLM Provider	Groq	Fast LLM inference
Default LLM	llama-3.3-70b-versatile	Model for answer generation
UI Framework	Chainlit	Chat interface
Styling	Custom CSS	Premium football-themed interface
UI Enhancement	Custom JavaScript	Hero section, prompt cards, animations, auto-scroll
Dataset Source	Wikipedia + curated summary file	Open-source knowledge base

5. System Architecture

The project uses a typical RAG architecture with additional reranking and UI enhancements.

```
Wikipedia Articles + Curated Summary File
↓
download_data.py / manual data files
↓
data/*.txt + data/sources.json
↓
ingest.py
↓
Text cleaning + token chunking
↓
HuggingFace embeddings
↓
ChromaDB vector database
↓
rag_chain.py
↓
Query expansion + candidate retrieval
↓
Cross-encoder reranking
↓
Best context chunks
↓
Strict prompt + Groq LLM
↓
app.py / Chainlit UI
↓
Answer + source evidence
```

6. Project Folder Structure

```

football-rag-chatbot/
├── architecture/
│   ├── rag_architecture.md
│   └── rag_architecture.png
├── data/
│   ├── *.txt
│   ├── sources.json
│   └── world_cup_awards_summary.txt
├── chroma_db/
├── public/
│   ├── styles.css
│   ├── custom.js
│   ├── fifa26-logo.png
│   ├── logo_dark.png
│   └── logo_light.png
├── screenshots/
│   ├── welcome-ui.png
│   ├── answer-with-sources.png
│   └── hallucination-test.png
├── tests/
│   └── test_questions.md
├── .chainlit/
│   └── config.toml
├── app.py
├── download_data.py
├── ingest.py
├── rag_chain.py
├── requirements.txt
├── .gitignore
├── engineering_report.md
├── model_comparison.md
└── README.md

```

7. Dataset and Knowledge Base Design

The knowledge base was built using public Wikipedia articles related to association football and the FIFA World Cup. The dataset was expanded from a small general FIFA dataset into a richer tournament-level World Cup knowledge base.

- Core football and FIFA World Cup overview articles.
- Every FIFA World Cup tournament article from 1930 to 2026.
- World Cup finals, records, statistics, top goalscorers, final goalscorers, hat-tricks, and own goals.
- Award pages such as Golden Ball, Golden Boot, Golden Glove, Best Young Player, and Fair Play Trophy.
- Curated tournament-level summary file for winners, runner-ups, final scorelines, top scorers, awards, and milestones.

7.1 Curated Summary File

A custom curated text file, `world_cup_awards_summary.txt`, was added to improve exact retrieval for tournament-level questions. This file contains clean, direct facts for each tournament, including winner, runner-up, final scoreline, top scorer, Golden Ball, Golden Glove, Best Young Player, Fair Play Award, and major milestones where available.

This approach avoids overcomplicating data extraction from table-heavy Wikipedia pages while improving answer quality for common World Cup questions.

8. Data Ingestion and Preprocessing

The ingestion pipeline is handled by `ingest.py`. Its responsibilities include:

1. Load text files from the data directory.
2. Read source metadata from `sources.json` where available.
3. Clean the text by removing unnecessary spacing and formatting noise.
4. Split documents into token-based chunks.
5. Attach metadata such as title, filename, source URL, chunk index, and total chunks.
6. Generate embeddings for each chunk.
7. Store the embedded chunks in ChromaDB.

8.1 Chunking Strategy

The project uses token-based chunking with overlap. This helps keep related context together while allowing long documents to be split into searchable pieces.

Parameter	Value	Reason
Chunk size	500 tokens	Large enough to contain meaningful football context
Overlap	50 tokens	Keeps continuity between adjacent chunks
Overlap ratio	10%	Balances retrieval quality and storage efficiency

9. Embeddings and Vector Database

The embedding model converts each text chunk into a numeric vector. The vector database stores these vectors so that user questions can be matched against semantically similar chunks.

- Embedding model: `sentence-transformers/all-MiniLM-L6-v2`.
- Vector database: ChromaDB.
- Similarity space: cosine distance.
- Persisted locally in the `chroma_db` directory.
- The database can be rebuilt by running `python ingest.py`.

10. RAG Pipeline Implementation

The main RAG logic is implemented inside `rag_chain.py`. The pipeline works as follows:

8. Receive the user question from `app.py`.
9. Expand football-specific query terms such as top scorer, Golden Boot, best player, Golden Ball, and scoreline.
10. Retrieve a larger candidate set from ChromaDB.
11. Attach retrieval score and source confidence metadata.
12. Rerank retrieved chunks using a cross-encoder reranker.
13. Format the best chunks into context for the prompt.
14. Send the context and question to the Groq LLM using a strict RAG prompt.
15. Return the answer, sources, and retrieval metadata to the Chainlit UI.

11. Retrieval Scoring, Query Expansion, and Reranking

11.1 Source Confidence

The system converts raw ChromaDB distance values into simple confidence labels. This makes source quality understandable for users without exposing technical scores.

Score Range	Confidence	Meaning
0.00 - 0.35	High	Strong semantic match
0.36 - 0.60	Medium	Moderate match
Above 0.60	Low	Weak or loose match

11.2 Query Expansion

Football users may use different words for the same concept. For example, top scorer and Golden Boot often refer to the same tournament award. Query expansion adds related terms before vector retrieval so the correct chunks are more likely to be found.

User phrase	Expanded meaning
top scorer	Golden Boot, leading goalscorer, most goals
best player	Golden Ball, player of the tournament
best goalkeeper	Golden Glove, Yashin Award
scoreline	final score, result, defeated, won
team of the tournament	All-Star Team, Dream Team

11.3 Reranking

The vector search retrieves a larger set of candidate chunks first. A cross-encoder reranker then compares the user question with each chunk and selects the most relevant chunks for the LLM.

```

User question
  ↓
Expanded retrieval query
  ↓
ChromaDB top candidate chunks
  ↓
Cross-encoder reranker
  ↓
Best top chunks
  ↓
Strict RAG prompt

```

12. Prompt Engineering and Hallucination Control

A strict RAG prompt instructs the model to answer only from retrieved context. This prevents the LLM from guessing or adding unsupported facts.

- Use only the verified source context.
- Do not use outside knowledge.
- Do not guess or predict.
- If the answer is missing, return a fixed fallback response.
- If partial information is available, answer the available part and clearly state which part is missing.

Fallback response:

The requested target info is missing from the provided dataset.

13. User Interface Design

The chatbot interface is built with Chainlit and customized using CSS and JavaScript. The goal was to create a premium football-themed interface rather than a plain default chatbot.

- Football-themed hero section.
- Dark premium visual style.
- Prompt cards for common questions.
- Custom football-inspired send and stop buttons.
- Animated background elements.
- Answer-focused auto-scroll behavior.
- Clean source evidence cards with source confidence labels.
- Responsive layout for different screen sizes.

14. Source Evidence Display

The UI displays a clean source section with each answer. Instead of showing technical chunk IDs, the system shows user-friendly information.

- Source title.
- Source confidence: High, Medium, or Low.
- Reason why the source was used.
- Source link.
- Short evidence preview.

15. Testing and Evaluation

Testing was performed using answerable, retrieval-quality, hallucination-control, out-of-domain, and edge-case questions.

Test Category	Example Question	Expected Behavior
Winner and final	Who won the 2010 FIFA World Cup and what was the scoreline?	Answer Spain 1-0 Netherlands after extra time if context is retrieved
Top scorer	Who was the top scorer in the 2018 FIFA World Cup?	Answer Harry Kane with 6 goals
Award	Who was the best player in the 2018 FIFA World Cup?	Answer Luka Modric / Golden Ball
Record	Who is the all-time top scorer in FIFA World Cup history?	Answer Miroslav Klose with 16 goals
Future prediction	Who will win the 2026 FIFA World Cup?	Return missing-information fallback
Out of domain	Explain quantum computing.	Return missing-information fallback

16. Model Comparison

A model comparison document was created to evaluate different Groq-supported models. The default model selected for the project is llama-3.1-8b-instant because it provides a strong balance of speed and answer quality for live demonstration.

Model	Provider	Purpose	Notes
llama-3.1-8b-instant	Groq	Default fast model	Best for live demo and quick testing
llama-3.3-70b-versatile	Groq	Higher-quality model	Better reasoning but slower
openai/gpt-oss-20b	Groq	Alternative open-weight model	Balanced option when available

17. Setup and Execution

17.1 Environment Setup

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

17.2 Environment Variables

```
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=llama-3.3-70b-versatile
HF_TOKEN=your_optional_huggingface_token_here
```

17.3 Run Pipeline

```
python download_data.py
python ingest.py
chainlit run app.py
```


18. Project Strengths

- End-to-end RAG system with modular Python files.
- Public, reproducible, and attributed data source strategy.
- Expanded FIFA World Cup coverage using tournament pages and curated facts.
- Strong hallucination control through strict prompt design.
- Improved retrieval using query expansion and reranking.
- User-friendly source evidence and confidence labels.
- Premium UI suitable for presentation and portfolio demonstration.
- Clear documentation through README, engineering report, test questions, architecture diagram, and model comparison.

19. Conclusion

The FIFA 26 Football RAG Chatbot is a complete knowledge-base chatbot project that demonstrates practical implementation of Retrieval-Augmented Generation. It combines data collection, preprocessing, embeddings, vector search, reranking, prompt engineering, source transparency, hallucination control, and UI customization.

The project is suitable for academic evaluation, bootcamp submission, and portfolio presentation because it shows both the engineering pipeline and the user-facing product experience. The final system can answer grounded FIFA World Cup questions while refusing to guess when information is not available in the dataset.